

AWE: Adaptive Agents for Dynamic Web Penetration Testing

Akshat Singh Jaswal*
Stux Labs
akshat@stuxlabs.com

Ashish Baghel*
Stux Labs
ashish@stuxlabs.com

Abstract—Modern web applications are increasingly produced through AI-assisted development and rapid no-code deployment pipelines, widening the gap between accelerating software velocity and the limited adaptability of existing security tooling. Pattern-driven scanners fail to reason about novel contexts, while emerging LLM-based penetration testers rely on unconstrained exploration, yielding high cost, unstable behavior, and poor reproducibility.

We introduce AWE, a memory-augmented multi-agent framework for autonomous web penetration testing that embeds structured, vulnerability-specific analysis pipelines within a lightweight LLM orchestration layer. Unlike general-purpose agents, AWE couples context aware payload mutations and generations with persistent memory and browser-backed verification to produce deterministic, exploitation-driven results.

Evaluated on the 104-challenge XBOW benchmark, AWE achieves substantial gains on injection-class vulnerabilities - 87% XSS success (+30.5% over MAPTA) and 66.7% blind SQL injection success (+33.3%) - while being much faster, cheaper, and more token-efficient than MAPTA, despite using a mid-tier model (Claude Sonnet 4) versus MAPTA's GPT-5. MAPTA retains higher overall coverage due to broader exploratory capabilities, underscoring the complementary strengths of specialized and general-purpose architectures. Our results demonstrate that architecture matters as much as model reasoning capabilities: integrating LLMs into principled, vulnerability-aware pipelines yields substantial gains in accuracy, efficiency, and determinism for injection-class exploits. The source code for AWE is available at: <https://github.com/stuxlabs/AWE>

Index Terms—Web Security, Large Language Models, Penetration Testing, Autonomous Agents

I. INTRODUCTION

The increasing popularity of AI assisted software development and the limited adaptability of traditional security tools have created a widening gap in the web security landscape. Most notably recent development trends of no-code platforms, automated code-generation assistants, and rapid deployment pipelines allow web applications to be made by developers with limited security expertise. This broadens the attack surface significantly while existing security tooling

remain stuck in pattern based detections and lack genuine reasoning capabilities.

Recent OWASP Top 10 data shows that every major category of web weakness ranging from injection flaws to access control failures and server-side request manipulation continues to appear across most real world applications [1]. Despite ongoing advancements in secure development practices, these vulnerability classes remain persistent. The widening gap between accelerated development and static defensive capabilities has created a massive challenge for modern web security assessment.

To address this growing mismatch, we introduce AWE (Adaptive Web Exploitation Framework), a memory-augmented multi-agent penetration testing system designed for autonomous, intelligent, and transparent vulnerability discovery. AWE aims to bridge the gap between traditional scanners and general-purpose LLM agents by combining domain-specific exploitation logic with large language models, enabling targeted, explainable, and scalable vulnerability discovery in modern web applications.

II. THREAT MODEL

A. System Model

We consider an automated black box vulnerability discovery system that interacts with modern web applications through standard HTTP interfaces. The system exercises application endpoints using both GET and POST requests and uses parameter placement to explore multiple input channels. The target applications resemble modern web applications that expose parameterized HTTP endpoints and perform server-side processing using common frameworks (PHP, Python, Node.js, Java). These applications may incorporate input validation, output encoding, and application-layer firewalls. The system has no privileged visibility into source code, runtime logs, or internal application state. All observations arise solely from HTTP responses and timing behavior. The attacker's automation maintains a persistent memory across probes, enabling adaptive exploration.

B. Attacker Capabilities and Goals

We assume an automated adversary that interacts with the application strictly through black-box HTTP requests. The attacker is realistic, constrained, and possesses the following capabilities:

*The authors contributed equally to this work.

- **Black-box interaction.** The attacker can craft arbitrary HTTP requests and observe responses, but lacks source-code access, server configuration details, or a privileged capabilities.
- **Authenticated probing.** When available through benign registration or low-privilege accounts, the attacker may authenticate to explore additional endpoints and restricted input channels.
- **LLM-assisted input generation.** The attacker employs commercial LLM APIs to synthesize context-aware payloads and adapt strategies based on prior observations, subject to cost-bounded operation. The LLM serves as a flexible generator of candidate attack inputs.
- **Time-bounded evaluation.** Each target endpoint is probed under a strict temporal budget (≤ 10 minutes), reflecting practical constraints imposed by rate limiting, detection risk, and LLM API costs.

The attacker’s goals are to identify injection-class vulnerabilities throughout controlled manipulation of inputs, and exploit abnormalities in application behavior (e.g., response timing, error structure, output differences) to infer server-side faults.

C. Trust Relationship

We assume the target application stack is uncompromised and behaves according to its implementation, although it may contain vulnerabilities. The hosting infrastructure and network fabric are also considered trustworthy from a security standpoint, providing no privileged access to the adversary. All attacker-controlled inputs (parameters, headers, cookies, request bodies) are considered potentially malicious, and dynamic content originating from the client side may also serve as a vehicle for exploit construction

D. Scope

Our work focuses on injection-centric vulnerabilities that can be discovered solely through black-box manipulation of application inputs. Within this scope, we consider attacks that exploit improper handling of attacker-controlled data across a broad spectrum of server-side interfaces. These include vulnerabilities such as cross-site scripting, SQL injection in its various forms, server-side template injection across widely used templating engines, command injection, file inclusion and path traversal, XML external entity expansion, server-side request forgery, and unauthorized object access when valid credentials are available. The scope therefore encompasses vulnerabilities whose exploitability emerges from observable differences in application behavior under controlled input perturbation.

Out of scope are vulnerabilities that cannot be meaningfully exercised or detected through black-box interaction alone. We do not consider network-level or protocol-level attacks, cryptographic weaknesses, or business-logic flaws that require semantic domain knowledge or multi-step reasoning beyond observable request–response behavior. Our focus is strictly on

vulnerabilities that arise from input processing behavior accessible to a realistic, resource-constrained adversary operating through standard HTTP interfaces.

III. BACKGROUND AND RELATED WORK

A. Traditional Automated Vulnerability Scanning

Dynamic Application Security Testing tools remain the popular automated method for identifying security flaws in modern web applications. Commercial systems such as Burp Suite [2], as well as open-source tools like OWASP ZAP [3], Nuclei [4], and sqlmap [5], rely primarily on signature-driven payload databases combined with heuristic pattern matching. These tools excel at detecting well-understood classes of injection vulnerabilities by replaying curated payloads across various input vectors but this strict pattern matching also embeds inherent limitations.

One significant flow is that signature and template based scanners are static and they cannot synthesize novel payloads or mutate attack strategies when confronted with nonstandard sanitization, application specific input handling, or adaptive WAFs. Also, the rigidity of pattern matching leads to false positives when benign behaviors resemble known signatures, and false negatives when exploitation requires multi-step probing or contextual reasoning. [6]. Specialized tools such as sqlmap for SQL injection show excellent domain-specific performance but lack generality across heterogeneous vulnerability families and vulnerabilities with dependent chaining. Collectively, these limitations highlight the difficulty of expressing dynamic attack reasoning within static scanners.

B. LLM-Based Penetration Testing Systems

Large language models have recently motivated systems that apply natural language to security assessment. PentestGPT [7] was the first well crafted attempt that proved LLMs can support human testers by structuring workflows, suggesting reconnaissance strategies, and making exploit logic. Although impactful, these systems function primarily as assistive agents: humans maintain the memory, perform validation, and execute tools. Subsequent research has explored autonomous operation through multi-agent orchestration. Frameworks such as AutoPT [8], AutoAttacker [9], CAI [11], and related multi-agent LLM systems [10], [12] couple LLM-driven controllers with command execution environments and reconnaissance tooling. These approaches automate selected penetration testing phases, but typically rely on unspecialized reasoning models and lack persistent memory for tracking authentication status, filter behavior, or previously attempted payloads, features essential for complex injections. MAPTA [12] represents a significant advancement in autonomous LLM driven penetration testing. It employs a three role multi-agent architecture in which a Coordinator agent performs high-level planning, Sandbox agents execute commands and scripts within an isolated per job Docker environment, and a Validation agent converts candidate exploits into verified proof-of-concepts through concrete execution. By coupling LLM-based reasoning with structured tool orchestration and evidence-gated PoC

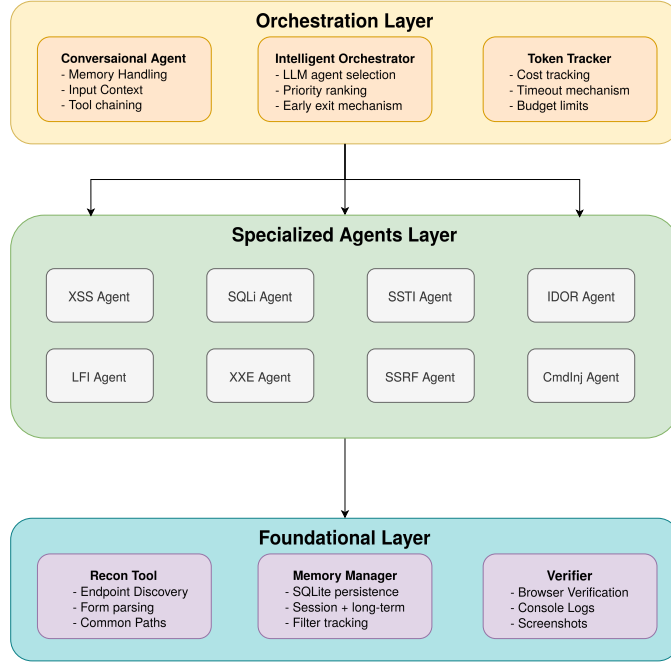


Fig. 1. AWE system architecture.

validation, MAPTA demonstrates that fully autonomous end-to-end web exploitation is feasible and establishes a strong baseline for agent-driven security testing.

C. Architectural Gaps in Existing Systems

Despite their individual strengths, both traditional scanners and existing LLM-based penetration testing systems share several fundamental limitations. Although LLM-based agents technically receive server-side feedback, they lack the domain-specific exploitation reasoning required to interpret that feedback and transform it into effective payload evolution. Practical exploitation often depends on subtle details (filter ordering, encoding quirks, type coercion behavior, template engine semantics, multi-parameter interactions etc.) that general-purpose LLM reasoning does not reliably model. As a result, existing systems tend to generate a small set of generic payloads, fail to recognize why they were blocked, and prematurely abandon the search rather than performing the iterative probing needed to infer sanitization logic or craft targeted bypasses.

Furthermore, most architectures do not maintain the rich contextual state necessary for multi-step exploitation, such as tracking which payload variants were attempted, how filter behavior has changed across requests, or which response features signal partial progress. Without structured memory and explicit state modeling, agents cannot build the multi-hop reasoning chains required for difficult injection classes.

Finally, the absence of domain-specialized probing techniques such as type confusion probing, template context shifting, timing-based inference, or controlled syntax fragmentation limits the ability of existing systems to move beyond superficial exploitation attempts.

These gaps reflect a broader challenge: current tools combine feedback and autonomous reasoning but lack the specialized, stateful, and iterative mechanisms necessary to convert raw feedback signals into precise, context-aware exploit generation.

IV. SYSTEM DESIGN

AWE is designed as an autonomous web exploitation system that integrates reconnaissance, domain specialized vulnerability analysis, and adaptive LLM reasoning under explicit resource constraints. Its architecture uses global orchestration to orchestrate vulnerability specific logic and has shared memory to enable systematic exploration of an application’s attack surface simultaneously ensuring that each component operates with clear and specific responsibilities. AWE consists of three architectural layers. The Orchestration Layer manages global states, coordinates agents and enforces budgetary constraints. The Specialized Agents Layer executes targeted exploitation strategies tailored to each vulnerability classes. The Foundation Layer provides common services such as hybrid payload generation, persistent memory, browser-based verification, and endpoint discovery/reconnaissance. The overall architecture is illustrated in 1.

A. Orchestration Layer

The Orchestration Layer manages the progression of a scan from initial reconnaissance through multi-step exploitation. Unlike traditional scanners, which treat each vulnerability class as an isolated test, AWE maintains a global exploitation context capturing the evolving state of the adversary. This includes information such as discovered inputs, observed server transformations, authentication status, prior payload attempts,

and successful exploitation steps. By storing this information in a unified state model, the orchestrator can reason about how new findings to adaptively influence the overall strategy. For example, upgrading to authenticated testing once credentials are obtained or suppressing redundant payload attempts based on previously observed failures.

At the center of this layer is the Intelligent Orchestrator, which mediates all interactions between components. It collects reconnaissance results, assesses the viability of different vulnerability classes, and selects appropriate agents to invoke. The selection process is done by a LLM that converts reconnaissance output into a prioritized execution plan. Rather than generating payloads directly, the LLM functions as an advisory mechanism, interpreting contextual cues such as reflected parameters, sanitization behavior or the presence of language specific templating constructs. The orchestrator therefore avoids the inefficiency of enumerating every agent and instead executes a minimal subset who meet the required preconditions.

The Orchestration Layer finally also enforces resource governance by monitoring token spend, runtime, and tool costs. This information steers scheduling so AWE can exit early after high-impact findings or scale back low-yield agents, keeping operations within practical limits while focusing depth where it matters.

B. Specialized Agents Layer

The Specialized Agents Layer embodies the domain knowledge required to navigate specific vulnerability classes. Each agent is implemented as a self-contained exploitation module that translates application behavior into vulnerability specific hypotheses and tests those hypotheses using structured procedures. Rather than relying solely on LLM reasoning, agents encode expert methodologies directly into their operational pipelines, ensuring predictable and reproducible behavior.

The XSS agent is the best example to explain the approach. The agent first conducts a multi-stage analysis beginning with parallel canary injections to map input reflection behavior to analyse for reflected XSS. The agent distinguishes among fine-grained DOM contexts, such as quoted and unquoted attributes, JavaScript string literals, or raw HTML insertion because the viability of subsequent payloads depends critically on contextual correctness. Following context identification, the agent performs targeted probing to infer server side filtering policies, including character level transformations, blocked tag families, and event handler restrictions. This information is packaged into a structured format passed to the LLM, which synthesizes payload candidates tailored to the discovered constraints. This design avoids the speculative or hallucinated vulnerabilities common in LLM-driven systems by grounding LLM creativity in precise contextual information and requiring definitive evidence of JavaScript execution. A visualization of this workflow is shown in 2.

Agents for SQL injection, server-side template injection, command injection, XXE, SSRF, IDOR, and LFI follow similar principles. The SQL injection agent combines deterministic

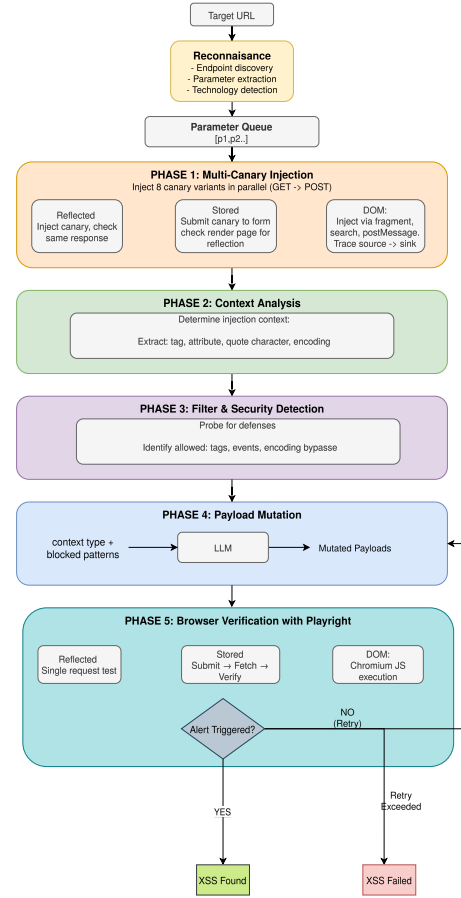


Fig. 2. Five-phase XSS detection pipeline.

payload sets with context inference derived from database error messages, backend fingerprinting, and observed query-structure patterns. It then applies controlled mutations to explore alternative execution paths or WAF bypasses. The SSTI agent deploys engine-specific probes to distinguish among popular templating frameworks and subsequently constructs exploit strings that reflect the internal semantics of the detected engine. The IDOR agent relies on authenticated differential testing, comparing access patterns across resource identifiers to detect authorization inconsistencies. In every case, the agents share the same design philosophy: integrate structured domain knowledge, reduce reliance on unconstrained LLM reasoning, and validate exploitability through concrete behavioral evidence.

C. Foundation Layer

The Foundation Layer provides a shared infrastructure which all agents use and operate upon. A core component is the Persistent Memory System, which combines short term scan states with long-term cross-target learning. Short-term memory prevents redundant attempts within the same engagement by tracking tried payloads and their outcomes, inferred filters, and agent-level progress markers. Long-term memory records domain-level features such as effective bypass patterns,

characteristic sanitization signatures, and historical payload success rates. This architecture allows AWE to integrate prior experience into future attacks and reduces unnecessary exploration, mirroring how a real adversary accumulates knowledge over repeated interactions with similar systems. The Browser Verification Engine is another core component that provides definitive exploit validation for vulnerability classes whose manifestation cannot be confirmed through HTTP responses alone. By executing payloads in a controlled browser environment, AWE observes concrete signals: script execution, DOM mutation, dialog triggers that are otherwise invisible to purely server-side testing. This eliminates entire categories of false positives and differentiates between theoretical and practically exploitable vulnerabilities.

Finally, services such as endpoint discovery, parameter extraction, and technology fingerprinting populate the initial attack surface and inform orchestrator decision-making. These components ensure comprehensive yet efficient enumeration of reachable interfaces and reduce unnecessary agent invocation by identifying the structural features most relevant to specific vulnerability classes.

Design Rationale

AWE’s architecture reflects three principled design choices:

- 1) **Specialization over generalized reasoning.** While LLMs excel at reasoning about semi-structured tasks, fine-grained exploitation requires domain-specific procedures that are more reliably implemented as dedicated state machines and inference pipelines.
- 2) **Stateful and memory-driven operation.** Modern exploitation depends on multi-step reasoning that spans numerous requests, input transformations, and contextual clues; a stateless scanner or unconstrained LLM agent cannot reliably maintain such context.
- 3) **Verification rather than speculation.** Every finding in AWE must be supported by concrete evidence—observable execution, differential behavior, or successful data extraction—ensuring that the system reports only vulnerabilities that a real adversary could exploit.

These design principles collectively enable AWE to operate as a practical, resource-bounded, and exploit-grounded autonomous penetration tester capable of discovering complex web vulnerabilities with high precision.

V. METHODOLOGY

This section outlines our evaluation methodology, including benchmark selection, baselines, model experiments, configuration, and metrics. The goal is to assess AWE’s effectiveness and efficiency under realistic attacker constraints while enabling reproducible comparison with state-of-the-art systems.

A. Benchmarks

We evaluate AWE on two complementary benchmarks to assess both competitive performance and controlled vulnerability analysis.

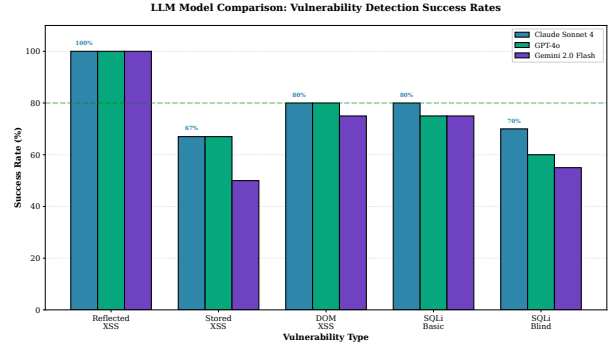


Fig. 3. Comparative performance of Claude Sonnet 4, GPT-4o, and Gemini 2.0 Flash across five vulnerability categories.

XBOW Benchmark: Our primary evaluation uses the XBOW benchmark [14], a curated suite of 104 vulnerable web applications spanning 26 vulnerability categories. Each challenge is deployed as an isolated container and embeds a hidden flag that is accessible only through a complete end-to-end exploit. XBOW provides substantial heterogeneity: vulnerabilities range from straightforward reflected XSS to multi-stage chains involving authentication, authorization, and context-specific sanitization bypasses. Injection-related categories constitute a majority of the benchmark mimicking the state of real world vulnerabilities. Challenges differ in exploitation complexity. Some are solvable through single-step injections, whereas others require the adversary to combine multiple findings, sequence authenticated requests, or adapt payloads to nontrivial server-side filters. This diversity makes XBOW a suitable testbed for evaluating AWE’s ability to perform adaptive exploitation at scale.

DVWA: For controlled model-selection experiments and fine-grained analysis of exploitation behavior, we use DVWA (Damn Vulnerable Web Application) [13]. DVWA offers repeatable vulnerability configurations and configurable security levels, enabling systematic testing across multiple difficulty regimes. We focus on reflected and stored XSS, DOM-based XSS, error-based SQL injection, and time-based blind SQL injection. Because the application is deterministic across runs, DVWA supports statistical comparison of model behavior under identical conditions. Each model is evaluated across multiple independent trials ($n=10$) per vulnerability type to obtain robust estimates of success rates and convergence behavior.

B. Baseline

We compare AWE against MAPTA in the XBOW Benchmark as it is the strongest publicly available autonomous penetration-testing framework. MAPTA adopts a general purpose multi-agent architecture in which a central LLM orchestrates reconnaissance, execution within an isolated sandbox, and exploit validation. MAPTA’s published evaluation reports a 76.9% solve rate on XBOW under generous compute and time budgets. Its architecture embodies the prevailing paradigm of broad, reasoning-centric agents, making it an